

Understanding the Transformer

A Technical Analysis of Attention Is All You Need

Independent Technical Review

Adrian — ML/AI Engineer

September 2026

This document is an original technical review. It explains and analyzes the ideas introduced by Vaswani et al. (2017); it is not a reproduction of the original paper.

Abstract

The Transformer architecture, introduced in Attention Is All You Need, replaced recurrence and convolution with attention as the central mechanism for sequence modeling. The result was a model that could process sequence elements in parallel during training, connect distant tokens through direct attention paths, and scale efficiently on modern hardware. This review reconstructs the architecture from first principles: why recurrent sequence models created computational bottlenecks, how scaled dot-product attention works, why multiple attention heads are useful, how positional information is injected, and how the encoder-decoder stack is trained.

The review also examines the engineering trade-offs behind the original design and explains why the Transformer became a foundation for modern language models. In particular, the same core ideas—token representations, positional information, attention, residual connections, normalization, feed-forward transformations, and autoregressive decoding—remain visible in contemporary large language model systems, even when the original encoder-decoder structure has been simplified or substantially modified.

1. Introduction

Sequence modeling asks a neural network to reason over ordered data. In natural language, order matters: the meaning of a word depends on its neighbors, syntax can span many tokens, and useful evidence may appear far from the current prediction target. Before the Transformer, recurrent neural networks (RNNs), gated variants such as LSTMs and GRUs, and convolutional sequence models were dominant approaches.

The Transformer proposed a different abstraction. Instead of forcing information to move through a recurrent chain or a limited convolutional receptive field, every position could directly interact with other positions through attention. The architecture therefore separated two problems that had often been entangled: representing token content and deciding which other positions were relevant.

The central engineering insight is as important as the mathematical one. A recurrent layer processes positions sequentially, while attention over a complete sequence can be expressed largely as matrix operations. On accelerator hardware, this difference has a major impact: matrix multiplication is highly parallelizable, whereas a recurrence introduces dependencies between successive time steps.

2. Background: Sequence-to-Sequence Learning

A sequence-to-sequence model maps an input sequence $x = (x_1, \dots, x_n)$ to an output sequence $y = (y_1, \dots, y_m)$. Neural machine translation is the canonical example: the input is a sentence in one language and the output is its translation in another.

A conventional encoder-decoder RNN first reads the source sequence and produces hidden states. The decoder then generates target tokens one at a time. Attention mechanisms improved this design by allowing the decoder to inspect a collection of encoder states rather than relying only on one compressed representation.

The Transformer takes the next step: attention is not merely an auxiliary component around recurrence. It becomes the primary mechanism used to exchange information between positions.

3. Why Move Beyond Recurrence?

- Sequential dependency: recurrent computation limits how many positions can be processed simultaneously.
- Long-range communication: information may need to travel through many recurrent steps.
- Hardware utilization: large matrix operations are generally easier to parallelize than step-by-step recurrent updates.
- Optimization depth: long computational paths can make learning relationships between distant positions harder.

Attention reduces the communication distance between any two positions to a single interaction. That does not make the computation free: full self-attention has quadratic dependence on sequence length. The important change is the shape of the computation, not the elimination of all computational cost.

4. Scaled Dot-Product Attention

Attention can be viewed as a weighted information-retrieval operation. Each token produces three vectors: a query, a key, and a value. Queries express what a position is looking for; keys describe what each candidate position contains; values carry the information that will actually be aggregated.

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$$

The dot product QK^T produces pairwise compatibility scores. Dividing by $\sqrt{d_k}$ prevents the scores from growing too large as the key dimension increases. The softmax converts the scores into normalized weights, and multiplying those weights by V produces a weighted combination of the value vectors.

The operation can be interpreted geometrically: attention builds a data-dependent mixing matrix. Unlike a fixed convolution kernel, the pattern of information exchange changes with the input itself.

5. Multi-Head Attention

A single attention operation produces one pattern of interaction. Multi-head attention instead projects the representations into several learned subspaces, performs attention independently in each head, concatenates the results, and applies a final projection.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

This gives the model multiple opportunities to represent relationships simultaneously. One head may specialize in local syntactic relationships, another may track longer-range dependencies, and another may behave differently again. These interpretations are useful intuitions rather than guarantees: the learned heads do not come with fixed human-readable roles.

6. Positional Encoding

Self-attention by itself is largely permutation-equivariant: if the same set of token vectors is reordered, attention has no inherent notion of the original order. Language, however, is strongly order-dependent. The Transformer therefore adds positional information to token representations.

$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{(2i/d_{\text{model}})})$$

$$\text{PE}(\text{pos}, 2i+1) = \cos(\text{pos} / 10000^{(2i/d_{\text{model}})})$$

The original architecture used deterministic sinusoidal positional encodings. Different frequencies allow positions to be represented through a structured set of periodic signals.

7. Transformer Encoder

The encoder is a stack of repeated layers. Each layer contains two principal sublayers: multi-head self-attention and a position-wise feed-forward network. Residual connections and layer normalization surround these transformations.

Conceptually, the encoder repeatedly performs two operations. First, tokens communicate with one another through attention. Second, each position applies a nonlinear transformation to its resulting representation. Repeating this process allows information to mix across the sequence while progressively transforming the representation space.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

The feed-forward block is applied independently at each position but uses shared parameters across positions. This separation between cross-token communication and per-token transformation is a useful conceptual lens for understanding the architecture.

8. Transformer Decoder

The decoder is also composed of repeated layers, but it contains an additional attention sublayer. A decoder layer includes masked self-attention, encoder-decoder attention, and a position-wise feed-forward network.

During training, a position must not be allowed to inspect future target tokens because doing so would reveal the answer. A causal mask therefore blocks attention to positions to the right of the current token.

The decoder also attends to the encoder output. In this sublayer, the decoder representation provides queries while the encoder representations provide keys and values.

At inference time, target tokens are generated sequentially. The model predicts a probability distribution over the vocabulary, selects or samples a token, appends it to the context, and predicts again.

9. Residual Connections and Normalization

Deep attention stacks depend heavily on stable information flow. Residual connections provide a direct path around each major transformation. Layer normalization stabilizes the representations passed between sublayers. Together, these components make it possible to train a deep stack without treating every layer as an isolated transformation.

This pattern became one of the most reusable architectural motifs in modern deep learning: a complex transformation is inserted into a residual pathway, with normalization used to control the distribution of activations.

10. Training Objective

For translation, the decoder models the conditional probability of the target sequence given the source sequence. The likelihood can be factorized autoregressively:

$$P(y \mid x) = \prod_t P(y_t \mid y_{(<t)}, x)$$

Training typically minimizes the negative log-likelihood, equivalent to token-level cross-entropy. During teacher forcing, the ground-truth previous target tokens are provided as decoder inputs while the model learns to predict the next token.

The original Transformer work demonstrated that an attention-only architecture could reach strong translation quality while training substantially faster than several recurrent alternatives.

11. Computational Trade-offs

The Transformer's advantage is not simply that attention is "faster." Its advantage depends on hardware, sequence length, model dimensions, implementation, and the competing architecture. Attention exposes large matrix operations that accelerators handle well, but the n^2 interaction matrix becomes increasingly expensive for long contexts. This tension is one reason later research explored sparse, linear, local, recurrent, and state-space alternatives.

12. Experimental Results and the Original Contribution

The original paper evaluated the Transformer on WMT 2014 English-to-German and English-to-French machine translation tasks. It reported strong BLEU scores and emphasized training efficiency relative to contemporary recurrent and convolutional sequence models. The exact numerical presentation varies across versions and reporting contexts, so the primary paper should be consulted when reproducing the historical benchmark.

The deeper contribution was architectural: the work showed that recurrence was not a prerequisite for high-quality sequence transduction. Attention could be the main mechanism through which a model built contextual representations.

13. Why the Transformer Changed AI

The Transformer created a general-purpose recipe for large-scale representation learning. Once token sequences could be processed with highly parallel operations, scaling model width, depth, dataset size, and compute became more straightforward. This aligned unusually well with the economics of modern accelerators.

The architecture also created a clean interface between context and computation. A token representation can query the rest of the sequence, gather relevant information, and pass the result through nonlinear transformations. This abstraction transfers naturally beyond translation to language modeling, multimodal systems, vision, speech, and many other domains.

14. From the Transformer to Modern LLMs

Many modern large language models use decoder-only Transformer variants rather than the original encoder-decoder architecture. The decoder stack remains attractive for autoregressive text generation because causal self-attention naturally matches next-token prediction.

Modern systems also modify many details: normalization placement, activation functions, positional representations, attention implementations, parameter sharing, tokenizer design, context-window mechanisms, and inference optimizations. Nevertheless, the core computational idea—learned token representations exchanging information through attention—remains recognizable.

During autoregressive generation, previously computed keys and values can be cached so the model does not recompute the entire prefix at every step. This is a major practical optimization for inference, although the cache itself consumes memory that grows with context length.

Memory-efficient attention kernels reduce the overhead of materializing intermediate attention matrices and improve hardware utilization.

15. Limitations and Trade-offs

- Quadratic attention: full self-attention becomes expensive as context length grows.
- Autoregressive decoding: generation still requires sequential token decisions.
- Memory pressure: activation storage and key-value caches can become substantial.
- Interpretability: attention weights are informative signals but should not automatically be treated as faithful explanations of model reasoning.
- Data and compute dependence: scaling the architecture does not eliminate the need for high-quality data, careful optimization, and evaluation.

The success of Transformers should therefore not be confused with universal optimality. The architecture is powerful because its computation maps well onto modern hardware and because learned attention provides flexible information routing. But those strengths come with clear costs at long sequence lengths and during large-scale inference.

16. Technical Analysis: What the Architecture Really Buys You

Architecture	Parallel training	Long-range path	Sequence-length scaling
--------------	-------------------	-----------------	-------------------------

RNN	Limited	Long	Roughly linear per step
CNN sequence model	High	Depends on depth/kernel	Roughly linear in positions
Self-attention	High	Direct	Quadratic interaction matrix

At a high level, the Transformer can be understood as a learned communication system. Each layer gives every position an opportunity to read from other positions, then transforms the collected information. Stacking layers repeatedly expands the kinds of relationships that can be represented.

Three ideas are especially important. First, content-dependent routing: attention weights depend on the current representations. Second, parallelizable computation: the core training operations can be organized into matrix multiplications. Third, residual composition: each layer can incrementally modify a shared representation rather than rebuilding it from scratch.

These properties help explain why the architecture scales. More layers provide more sequential transformations; more dimensions provide a larger representation space; more data exposes the model to more statistical structure; and more compute allows optimization over larger parameter spaces.

17. Reproducibility Notes

A minimal educational implementation can be built from the equations in this review. The core components are: token embeddings, positional information, scaled dot-product attention, multi-head attention, feed-forward networks, residual connections, layer normalization, a causal mask for the decoder, and a final vocabulary projection.

A useful reproduction workflow is to validate each component independently. First verify tensor shapes. Then compare attention scores against a small hand-computed example. Next verify that the causal mask prevents information flow from future positions. Finally train a very small model on a toy sequence task before attempting a real translation dataset.

For historical reproduction of the 2017 results, use the original paper and its reported dataset preprocessing, model configurations, optimization schedule, and evaluation protocol rather than treating this review as a benchmark specification.

18. A Compact Mental Model

If you remember only one mental model, think of a Transformer layer as: look around → gather useful information → transform it → keep the old representation available. Attention performs the looking and gathering; the feed-forward network performs the transformation; residual connections preserve an information highway; normalization helps stabilize the stack; positional information tells the model where tokens occur.

19. Conclusion

Attention Is All You Need was a landmark because it challenged an assumption that had shaped sequence modeling for years: that recurrence was necessary to model ordered sequences. The Transformer showed that attention could serve as the dominant mechanism for contextual interaction while exposing computation to much greater parallelism.

Its lasting influence comes from the combination of a relatively simple mathematical primitive with an architecture that scales effectively. Modern language models have evolved far beyond the exact 2017 configuration, but their foundations remain closely related to the original ideas: token representations, positional structure, attention-based communication, nonlinear transformation, residual pathways, and autoregressive prediction.

For an ML/AI engineer, the Transformer is therefore worth learning at two levels. At the first level, understand the equations and tensor shapes. At the second, understand the engineering trade-offs—memory, compute, parallelism, numerical stability, inference latency, and scaling. The second level is where the architecture becomes more than a paper: it becomes a reusable way of thinking about modern AI systems.

Architecture comparison

References

- [1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems 30.
- [2] Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. ICLR.
- [3] Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to Sequence Learning with Neural Networks. NeurIPS.
- [4] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL.
- [5] Brown, T. B. et al. (2020). Language Models are Few-Shot Learners. NeurIPS.

Prepared as an original technical review for Adrian's ML/AI research portfolio.